

Chapter 26: Estimation for Software Projects (Part 2)

Lecture Title



Subject: Software Engineering



Program: BTech Computer Science and Engineering



Duration: 1 Hour

Advanced Software Project Estimation

Beyond Basic Decomposition

The Evolution of Estimation:

➤ Last lecture: Breaking projects into functional pieces.

The new questions:

- What if we only have use cases?
- What if we need to leverage data from thousands of past projects?
- How do we adapt to Agile and WebApp environments?
- What if the best choice is not to build it at all?

Today's Focus: Completing the estimation toolkit with empirical models, specialized techniques, and strategic decision-making.



Learning Objectives

What You Will Master

By the end of this lecture, you will be able to:

- **Calculate by Use Case:** Apply use-case-based estimation utilizing Use Case Points (UCP).
- **Leverage Historical Data:** Understand and apply empirical estimation models, specifically COCOMO II and the Software Equation.
- **Adapt to Modern Contexts:** Tailor your estimation techniques for Object-Oriented, Agile, and WebApp projects.
- **Make Strategic Choices:** Perform a rigorous 'Make vs. Buy' decision using decision tree analysis.
- **Assess External Partners:** Evaluate outsourcing options and objectively weigh their trade-offs.

Estimation with Use Cases

Leveraging Early Project Artifacts

The Premise: Often, we don't have detailed functionality or lines of code mapped out early in a project. We do have Use Cases from the requirements gathering phase.

The Metric: Use Case Points (UCP). A sizing metric similar to Function Points, but calculated based on the number and complexity of Actors and Use Cases.

The Five-Step Process:

1. **Classify the Actors**
(Simple, Average, Complex).

2. **Classify the Use Cases**
(Simple, Average, Complex).

3. **Calculate the Unadjusted Use Case Points (UUCP).**

4. **Apply the Technical Complexity Factor (TCF).**

5. **Apply the Environmental Complexity Factor (ECF).**

The UCP Classification Tables

Weighing Actors and Use Cases

1. Actor Classification (Who is interacting?)

Type	Description	Weight
Simple	External system with a well-defined API.	1
Average	A human user interacting via GUI or command line.	2
Complex	A human user interacting via a complex interface or across multiple systems.	3

2. Use Case Classification (How complex is the task?)

Type	Description	Weight
Simple	1 to 3 transactions (steps).	5
Average	4 to 7 transactions (steps).	10
Complex	More than 7 transactions (steps).	15

Calculating the Effort

Moving from Points to Hours

Calculate Unadjusted Points (UUCP):

UUCP = (Sum of Actor Weights)
+ (Sum of Use Case Weights)

Apply Technical Complexity (TCF):

(13 technical factors rated 0-5)

TCF = **0.6** +
(Sum of ratings × 0.01)

Apply Environmental Complexity (ECF):

(8 team/environment factors rated 0-5)

ECF = **1.4** + (**-0.03**
× Sum of ratings)

Calculate Final Adjusted UCP:

Adjusted UCP = **UUCP** × **TCF** × **ECF**

Convert to Effort:

Effort = **Adjusted UCP** × **Productivity Factor**
(Productivity factor is historically 20-40 person-hours per UCP)

Online Banking System Example

Applying UCP to a Real Project

Scenario: We are estimating an online banking portal.

1. Actors (Weight = 7)

Customer (Avg: 2)
Teller (Avg: 2)
Admin (Avg: 2)
Payment Gateway (Simple: 1)

2. Use Cases (Weight = 45)

Login (Simp: 5) Pay Bill
Balance (Simp: 5) Statement
Transfer (Avg: 10) Add User
Pay Bill (Avg: 10) (Avg: 10)

Complexity Factors

TCF (Average Technical Complexity) = 0.95
ECF (Average Team Experience) = 0.80

Base Calculation: $UUCP = 7 + 45 = 52$

Final Effort & Result

Adjusted UCP = $52 \times 0.95 \times 0.80$
= 39.5 UCP

Effort = $39.5 \times 25 \text{ hours/UCP}$
= 987.5 hours

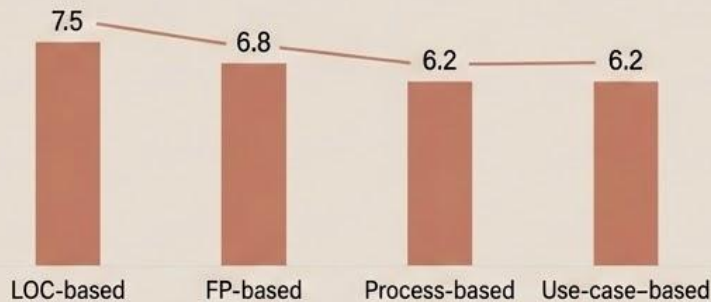
Result: $987.5 / 160$
= 6.2 person-months

The Golden Rule of Estimation



Reconciling Estimates: Triangulation for Confidence

Never rely on a single estimation technique.



Analysis:

- Because all four estimates cluster closely (between 6.2 and 7.5), our confidence in this estimate is extremely high.
- What if they diverge? If LOC said 12 and UCP said 4, we must stop, investigate our assumptions, and reconcile.

Finalizing the Number:

Calculate a weighted average, giving more weight to the techniques your specific organization has the most historical success with.

Empirical Estimation Models

Leveraging Historical Project Data

The Core Concept:

Instead of guessing or breaking a project into tiny pieces, we use mathematical formulas derived from analyzing thousands of past software projects.



$$\text{Effort} = A \times (\text{Size})^B \times M$$

- **A & B:** Empirically derived constants based on constants based on industry data.
- **Size:** Project size metric.
- **M:** A multiplier based on specific project characteristics (e.g., team skill, tool quality).

Types of Models:

- **Single-variable:** Effort is based purely on project size.
- **Multi-variable:** Incorporates size, people, process, and product traits.
- **Dynamic:** Accounts for how resources and time vary throughout the project lifecycle.

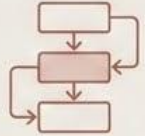


The Industry Standard

The COCOMO II Framework

The Constructive Cost Model

Origins: Originally developed by Dr. Barry Boehm at USC; updated to COCOMO II to account for modern iterative, object-oriented, and reuse-heavy development.



The Three Levels of Estimation:

Level	Description	When to Use It
Application Composition	For projects built with high reuse, existing components, and GUI builders.	Early prototyping phase.
Early Design	Provides rough estimates before detailed architecture is mapped out.	Right after scope definition.
Post-Architecture	The most detailed model; uses comprehensive cost drivers.	After the software architecture is fully defined.

The Math of COCOMO II

The Post-Architecture Model | Variables and Cost Drivers

$$\text{Effort} = 2.94 \times (\text{Size})^B \times M$$

Variables:

- **Size:** Measured in Thousands of Lines of Code (KLOC) or Function Points converted to KLOC.
- **B (The Exponent):** Ranges from 1.1 to 1.3 based on project characteristics (a higher number means a more complex, non-linear project).

M (The Multiplier): The product of 17 distinct 'Cost Drivers' categorized into four areas:

Product: Reliability requirements, database size, complexity.

Platform: Memory constraints, execution time limits.

Personnel: Programmer capability, analyst experience.

Project: Use of modern tools, schedule compression, process maturity.

The COCOMO II in Action

A Practical Calculation | Crunching the Numbers

The Scenario

- **Size:** 50 KLOC
- **Exponent (B):** 1.15 (A nominal/average project)
- **Multiplier (M):** 1.0 (Assuming all 17 cost drivers are average)

The Calculation

$$\text{Effort} = 2.94 \times (50)^{1.15} \times 1.0$$

$$\text{Effort} = 2.94 \times 88.5$$

Effort $\approx$ 260 person-months

The Advantages of COCOMO II

Extensively calibrated using real industry data.

Accounts for a massive array of specific project factors.

Highly tailorable (organizations can recalibrate the A constant using their own internal data).

The Software Equation

The Time-Effort Trade-off

Origins & Insight

- Developed by Lawrence Putnam, based on the Rayleigh curve of staffing over time.
- The relationship between size, effort, and time is heavily non-linear. Reducing the project schedule requires dramatically more effort.

$$\text{Effort} = \frac{\text{Size}}{(\text{Project Duration})^{4/3}}$$

The Reality Check (100 KLOC Project)

Baseline
(24 Months)
 $\frac{100}{(24)^{4/3}} \approx 1.04$
(scaled effort)

Compressed
(12 Months)
 $\frac{100}{(12)^{4/3}} \approx 2.63$
(scaled effort)



The Result & Implication

Cutting the schedule in half doesn't double the effort; it increases it by 2.5x.

Mathematically confirms Brooks' Law—adding more people to a late project just makes it later.

Estimating Object-Oriented Projects

Moving Beyond Traditional Metrics

The Challenge

Traditional metrics like Lines of Code (LOC) or Function Points don't always map cleanly to Object-Oriented (OO) design, where the fundamental units are classes and objects.

Modern Approaches for OO

- **Use Case Points:** Highly effective because OO design often starts directly from use cases.
- **CK Metrics:** Using Chidamber & Kemerer metrics as complexity indicators.
- **Object Points:** Sizing based on tangible interface elements.

The Challenge

Traditional metrics like Lines of Code (LOC) or Function Points don't always map cleanly to Object-Oriented (OO) design, where the fundamental units are classes and objects.

Object Point Estimation Steps

1 Count the number of screens, reports, and 3GL components required.

2 Classify each item as simple, medium, or complex.



3 Assign & Sum standard weights to each class to get total Object Points.

4 Apply a productivity factor based on specific developer experience.

Estimation in Agile Development

Embracing Relative Sizing

Why Agile is Different

- There are no massive, detailed upfront requirements documents.
- Work is estimated incrementally as it is pulled into the backlog.

Core Agile Estimation Techniques



Story Points

A relative measure of size and complexity (not hours). You estimate new stories by comparing them to a known reference story.



Planning Poker

A consensus-building game where team members estimate using numbered cards, discussing differences until agreement is reached.



Velocity

A historical measure of how many Story Points the team successfully completes per iteration (Sprint).



Ideal Days

Estimating based on how many days a task would take with zero interruptions or meetings.

The Agile Process in Action

Agile Forecasting: From Story Points to Sprints

The Estimation Process

- 1 Write User Stories:** Break features down into functional requirements.
- 2 Estimate with Fibonacci:** Assign Story Points using the sequence (1, 2, 3, 5, 8, 13, 21...) to reflect that larger tasks carry inherently higher uncertainty.
- 3 Measure Velocity:** Track how many points the team completes in the first few iterations.
- 4 Forecast:** Total Iterations = Total Story Points / Velocity

Scenario Example

Total project backlog = 120 Story Points

Measured Team Velocity = 20 Story Points per sprint

Sprint length = 2 weeks

 **Forecast:**

$$\frac{120}{20} = 6 \text{ sprints}$$

(or 12 calendar weeks)

Estimating WebApp Projects

Content, Navigation, and Rapid Cycles



Content-based

- They are highly content-heavy, evolve rapidly, and typically operate on very short release cycles.

Hybrid Estimation Approaches



Function-based

Count the specific backend user functions or transactions (similar to Use Cases).



Navigation-based

Count the distinct links, navigational paths, and routing logic required.

A Practical WebApp Example

Static pages: 10 pages @ 1 hr/each = 10 hours

Dynamic pages: 15 pages @ 3 hrs/each = 45 hours

Images/graphics: 20 assets @ 0.5 hrs/each = 10 hours

User functions: 8 functions @ 8 hrs/each = 64 hours

Total Estimated Effort: 129 hours (~3.2 person-weeks)

The Make/Buy Decision

Allocating Capital and Effort

The Core Question:

- Build the software entirely in-house?
- Buy existing Commercial Off-The-Shelf (COTS) software?
- Outsource the development to an external vendor?

The Reality:

Software engineering is not just about writing code; it is about delivering value.

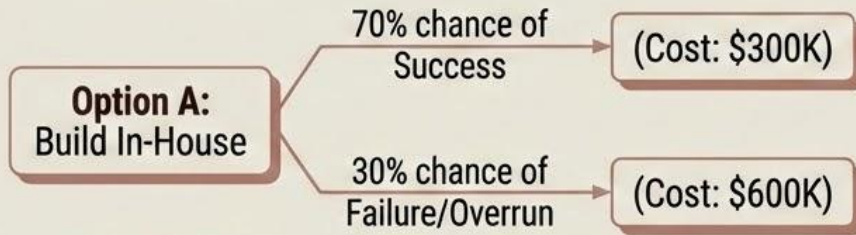
Sometimes, writing the code yourself is the most expensive and risky path to delivering that value.



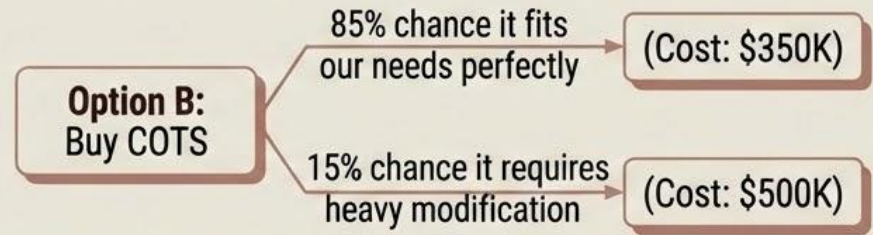
Quantifying the Choice

Decision Tree Analysis: Calculating Expected Cost Under Uncertainty

- **What it is:** A quantitative technique used to compare different alternatives when the outcomes are uncertain.
- **The Math:** We calculate the Expected Cost by multiplying the estimated cost of an outcome by its statistical probability.



Expected Cost:
 $(0.70 \times \$300K) + (0.30 \times \$600K) = \$390K$



Expected Cost:
 $(0.85 \times \$350K) + (0.15 \times \$500K) = \$372.5K$



Conclusion: Statistically, Buying is the safer financial choice (\$372.5K vs. \$390K).

Beyond the Numbers: Qualitative Factors in the Make/Buy Decision

Balancing Cost with Strategic Value



Factor	Build (In-House)	Buy (COTS)	Outsource
Control	High	Low	Medium
Time-to-Market	Slow	Fast	Medium
Strategic Value	High	Low	Medium
Cost Structure	Variable (Hours)	Fixed (License)	Variable (Contract)
Maintenance	Internal Team	Vendor	Contractor

The Third Option: Outsourcing Development

Leveraging External Expertise

Definition: Contracting an external organization to develop the custom software for you.

The Three Geographies



Domestic

- Within your same country
- Easiest communication, highest cost



Offshore

- In a different country
- Highest cost savings, severe communication/time-zone challenges



Nearshore

- In a neighboring country
- Balances cost savings with favorable time zones

The Trade-offs



Pros

- Significantly lower labor costs
- Instant access to specialized tech skills you lack
- Allows your internal team to focus on core business



Cons

- Severe communication barriers
- Cultural differences
- Loss of day-to-day control
- Intellectual property (IP) risks

Outsourcing Best Practices

Mitigating the Risks

If the decision tree points to outsourcing, project managers must enforce strict guardrails:



Ironclad Contracts: Write highly detailed contracts and Service Level Agreements (SLAs) regarding code quality and delivery dates.



Internal Oversight: Maintain strong in-house project management. Never outsource the management of the project, only the execution.



Continuous Auditing: Conduct regular code reviews and security audits; do not wait until the final delivery.



Protect IP: Ensure strict legal protections for your company's intellectual property and data.



Exit Strategy: Plan thoroughly for knowledge transfer so your internal team can maintain the software once the contract ends.

The Full Estimation Toolkit

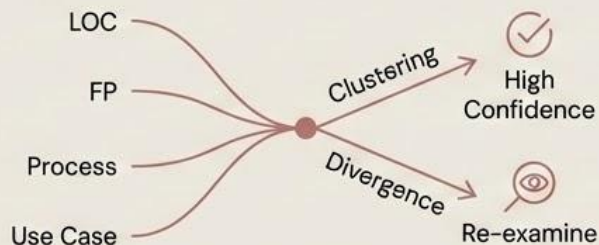
Synthesizing the Methods | From Use Cases to Reconciliation

Use-Case-Based Estimation

- Calculates effort using Use Case Points (a combination of Actor complexity and Use Case complexity).
- Adjusted using Technical (TCF) and Environmental (ECF) factors.

The Golden Rule: Reconciliation

- Never rely on a single method.
- Reconcile estimates across multiple techniques (LOC, FP, Process, Use Case).
- Clustering = High confidence.
- Divergence = Stop and re-examine underlying assumptions.



Empirical Estimation Models

The Power of Industry Data



COCOMO II (Constructive Cost Model):

$$Effort = A \times (Size)^B \times M$$

- Adjusts for 17 specific cost drivers.
- Application Composition → Early Design → Post-Architecture.



The Software Equation:

Highlights the non-linear relationship between size, effort, and time.



The Time-Effort Trade-off:

Artificially compressing a schedule results in an exponential explosion of required effort and cost.

Specialized Estimation

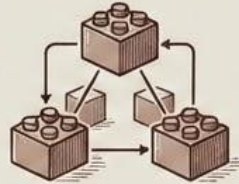
Adapting to the Environment

Estimation is not one-size-fits-all; the technique must match the methodology:



Agile Development

- Drops hours in favor of relative sizing.
- Uses Story Points, Planning Poker, and Velocity-based forecasting.



Object-Oriented (OO)

- Focuses on structural elements.
- Uses Object Points and Use Case Points.



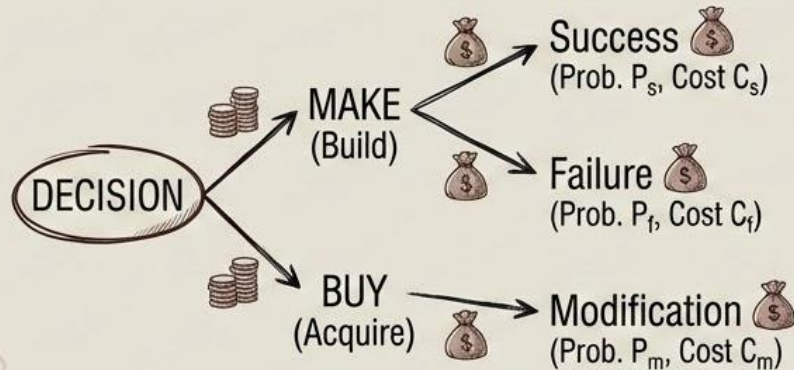
Web Applications

- Requires a hybrid approach due to rapid cycles and media density.
- Estimates based on Content (static/dynamic), Functions, and Navigation paths.

The Make/Buy Decision

To Build or Not to Build

The Decision Tree: A Quantitative Tool



A quantitative tool to calculate the Expected Cost of building versus buying by factoring in the statistical probability of success, failure, or modification.

Qualitative Factors: Beyond the Cost

Cost is only half the equation. You must also consider:



- Degree of Control needed.



- Required Time-to-market.



- Strategic Value of the technology.



- Risk tolerance (especially when considering Outsourcing).

The Purpose of Estimation

A Guide, Not a Guarantee

Estimation is not about **perfect prediction**—it's about **reducing uncertainty** enough to make **good decisions**. Use multiple techniques, involve the people doing the work, and always leave room for the unknown.

